

- Covariant Types
- Factory Method
- Abstract Factory

- Covariant Types

→ We can return parent or any of its child in RTP.

```
class Student {
```

```
class IntelligentStudent  
    int iq;
```

```
public Student clone () {
```

```
public IntelligentStudent  
    clone () {
```

3

3

3

Factory Method:

↳ mass productⁿ

```
class UserService {
```

C/I/AC
✓✓

```
Database db; ← class attr;
```

```
Query q;
```

C/I/AC
✓✓

```
public User saveUser(User u) {
```

```
String q = "INSERT ...";
```

```
Query query;
```

```
if (db instanceof MySQL) {
```

```
    query = new MySQLQuery();
```

OCP X

SRP X

```
}
```

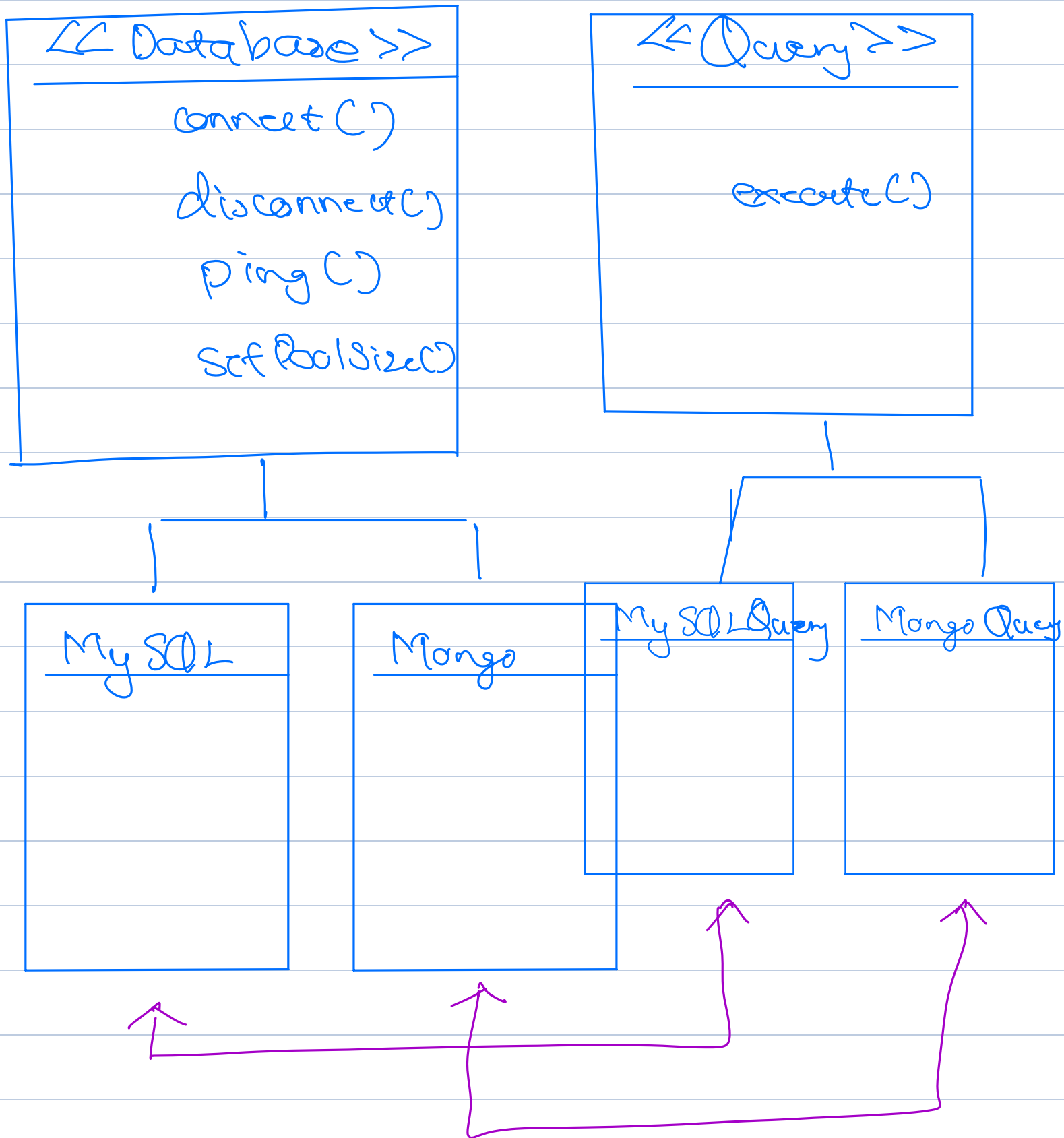
```
else {
```

```
    query = new MongoDBQuery();
```

```
    query.execute();
```

```
}
```

Original Diagram:

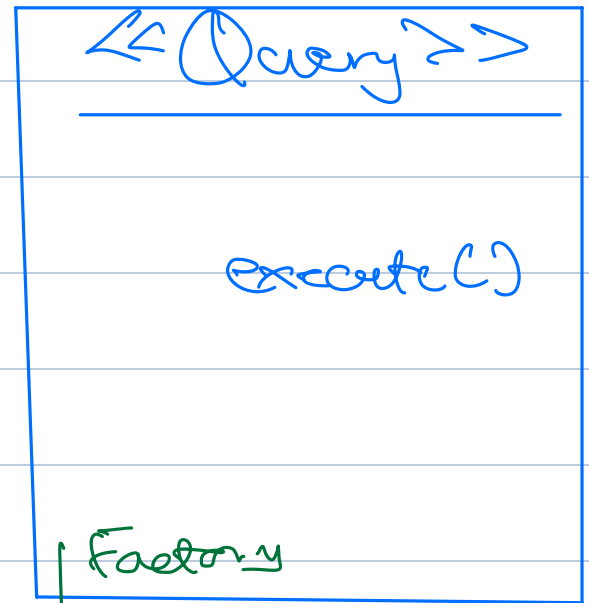
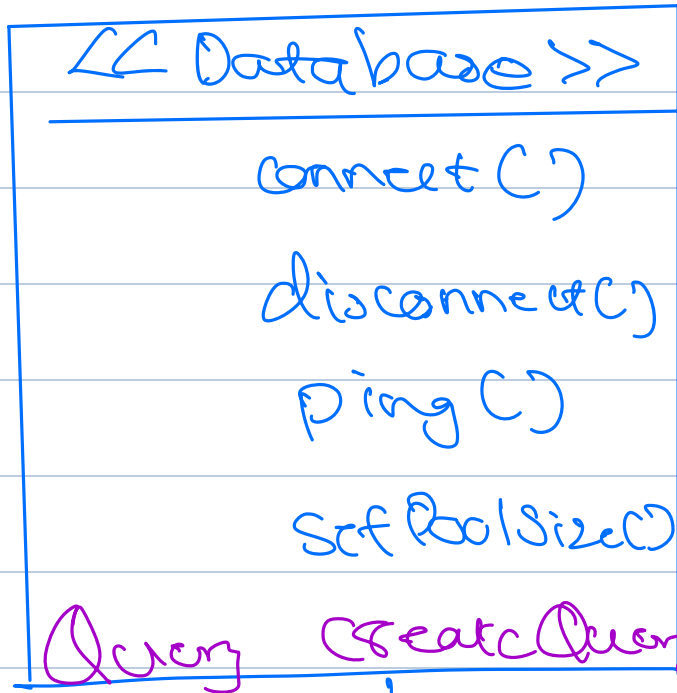


Problems with current design:

Client is responsible for creating a relevant query object.

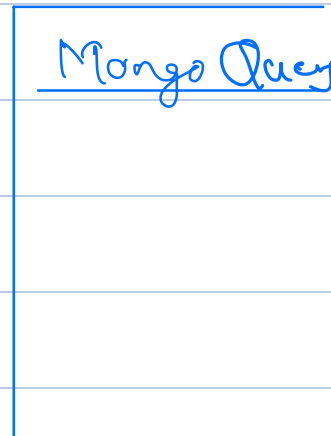
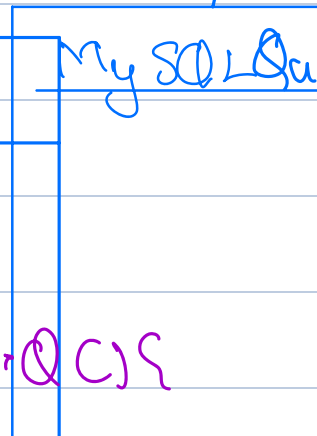
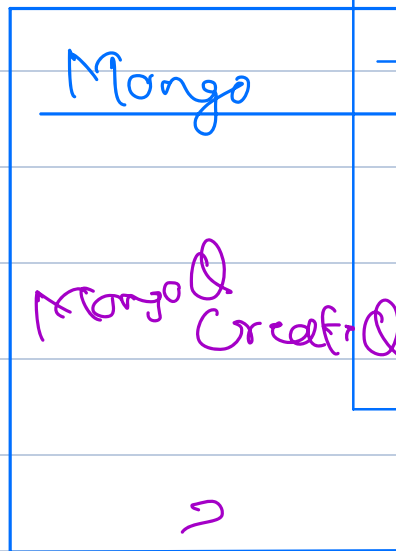
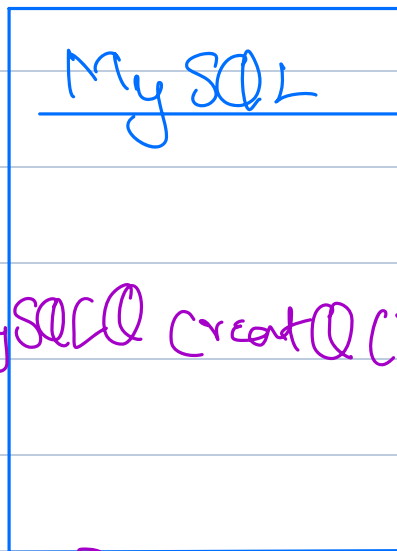
Updated Diagram:

Abstract Product



Factory Methods

Query createQuery()



MySQL createQuery()

Mongo createQuery()

3

2

Concrete Products

Updated Code:

```
Class UserService {
```

```
    Database db;
```

```
    public User saveUser (User u){
```

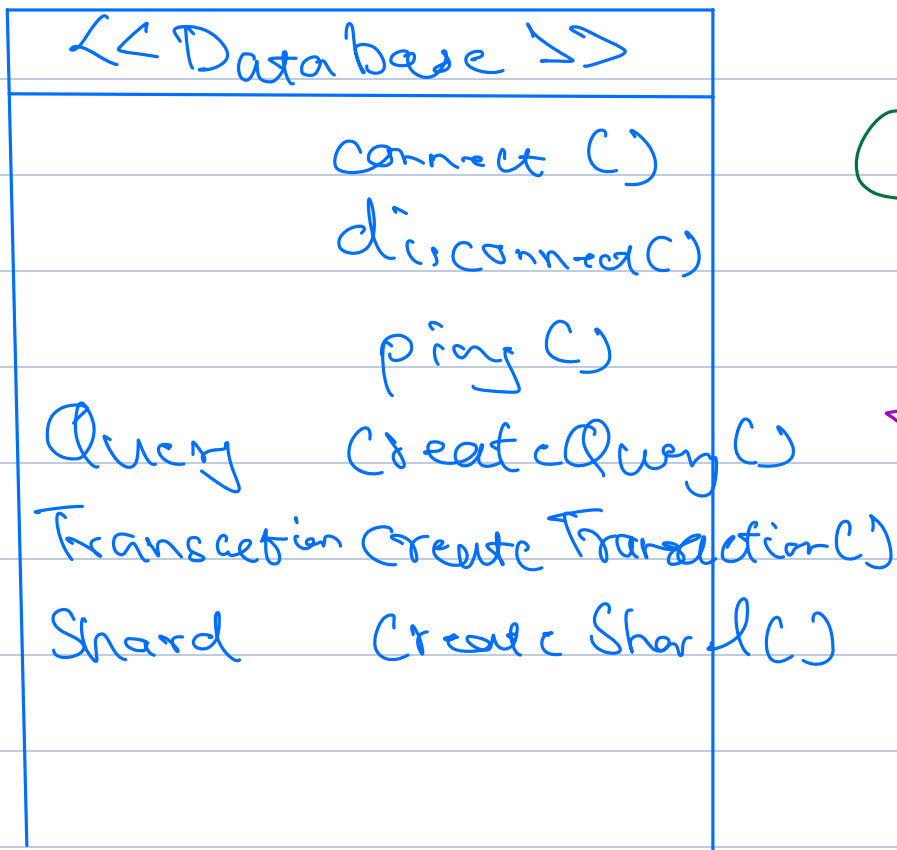
```
        String q = "INSERT ...";
```

```
        Query q = db.createQuery ();
```

```
        q.execute ();
```

Factory Method: is to delegate object creation from client to relevant class.

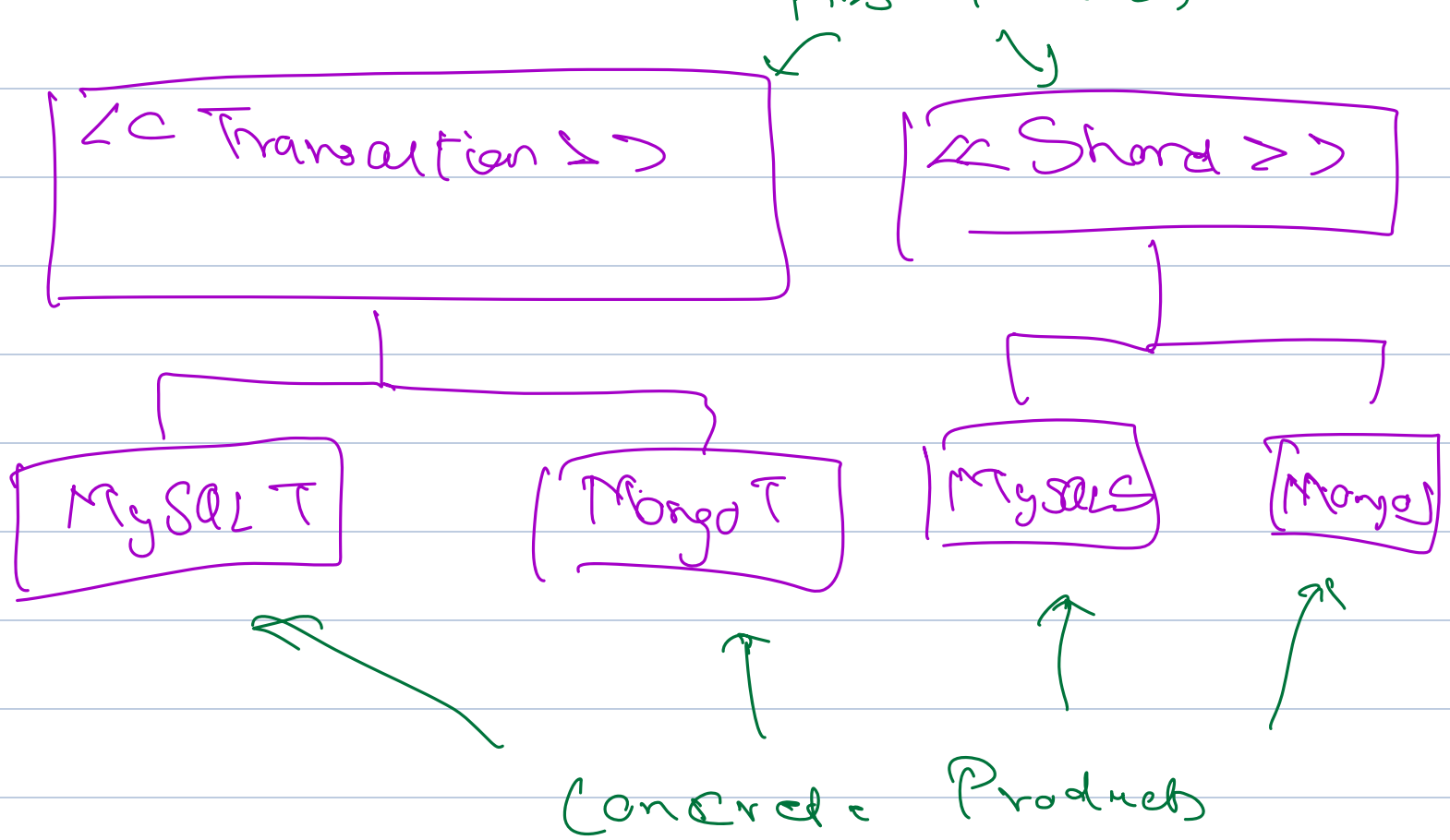
Abstract Factory:



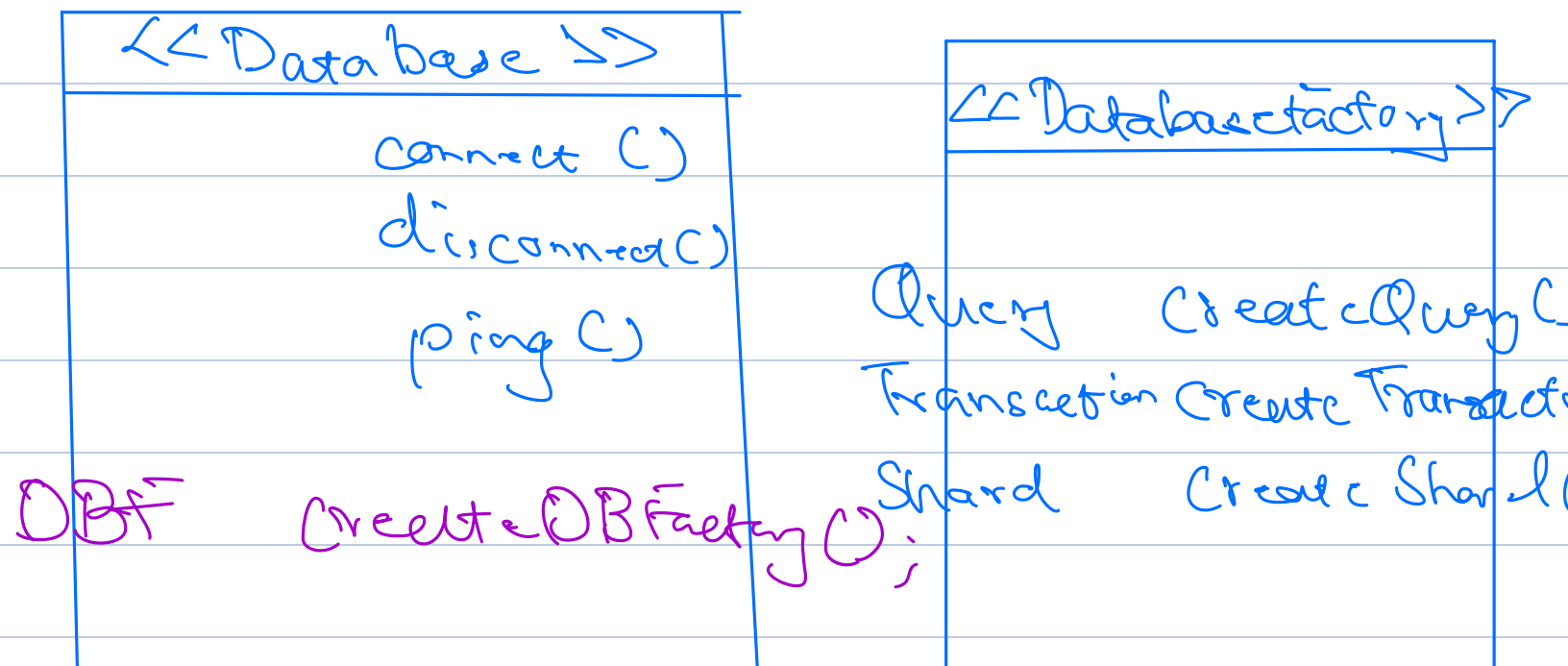
- Responsibilities:
- ① Expose methods related to a DB
 - ② Multiple Factory Methods

} Factory Methods

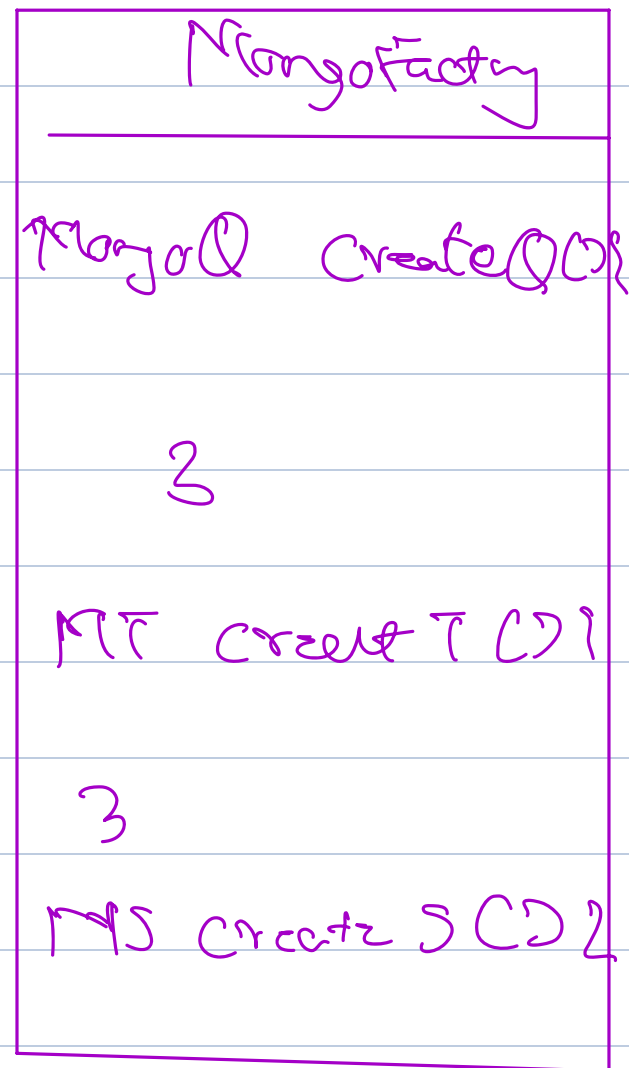
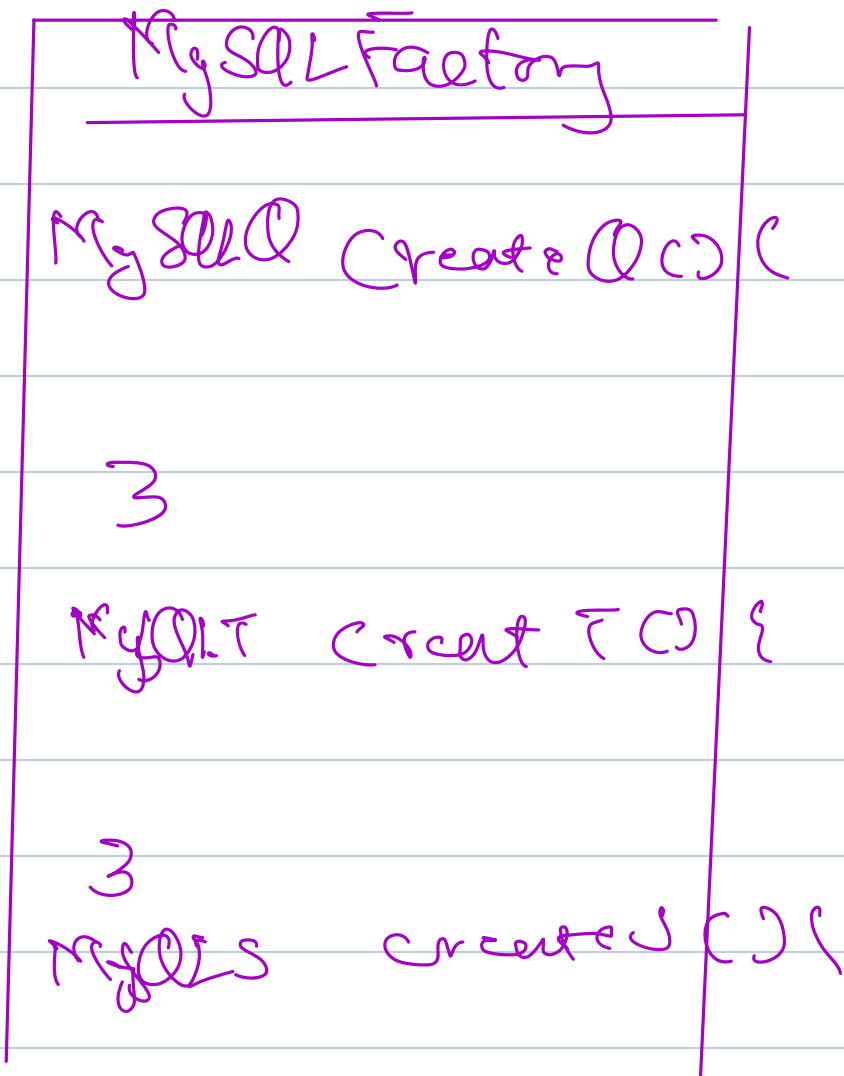
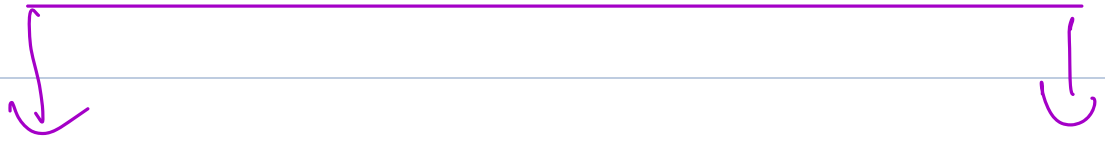
Abstract Products



The current interface has 2 responsibilities. To fix this we will break this interface:



↑
Resp: to hold all
FMS



3

Client code:

```
class UserService {
```

```
    Database db;
```

```
    public User saveUser (User u) {
```

```
        String q = "INSERT ....";
```

```
        DBF dbf;
```

```
        if (db is instance of MySQL {
```

SRRX

```
            dbf = new MySQLFC();
```

OCPX

```
        } else {
```

```
            dbf = new MongoDBFC();
```

```
        }
```

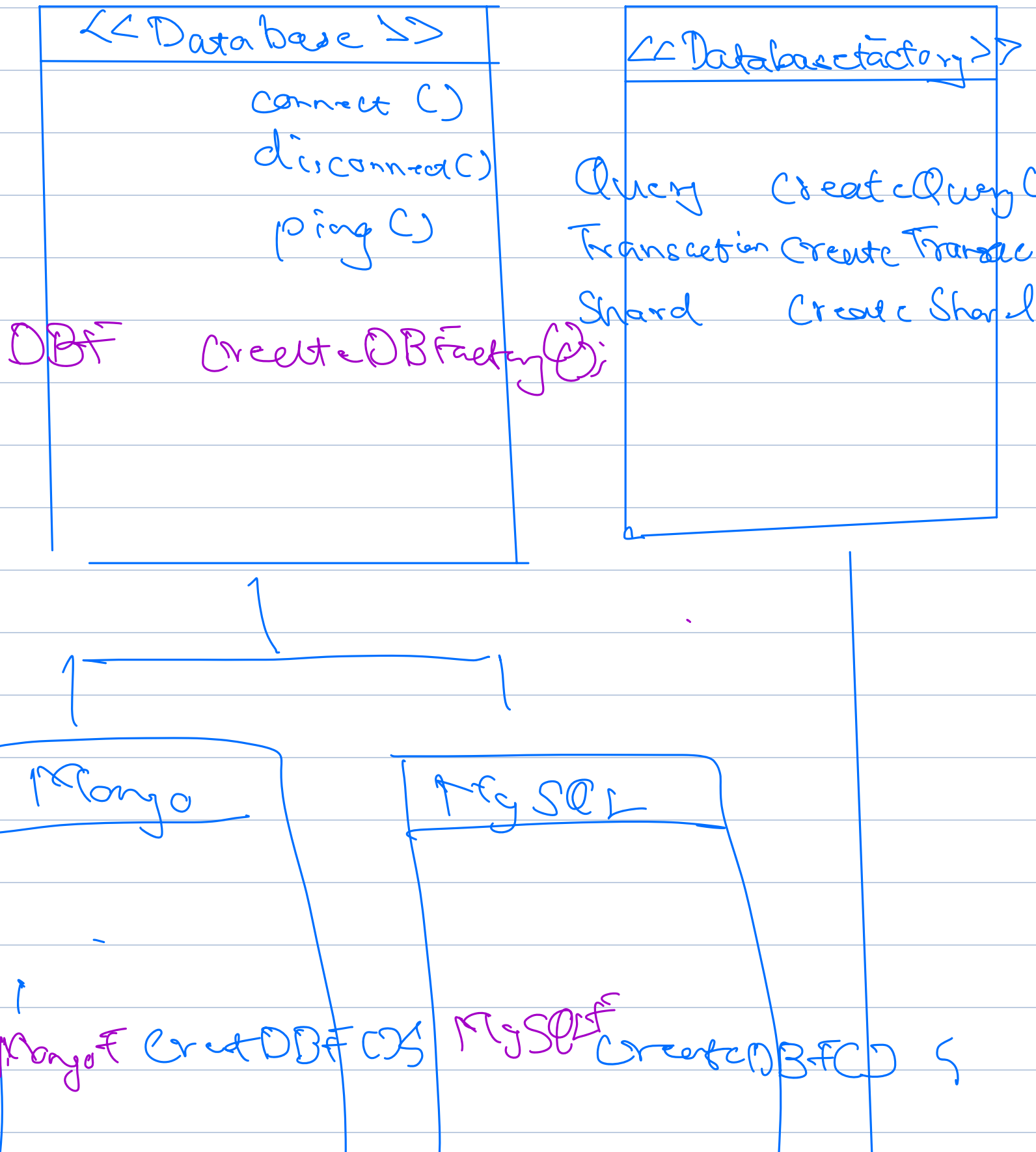
```
        Query q = dbf.createQuery();
```

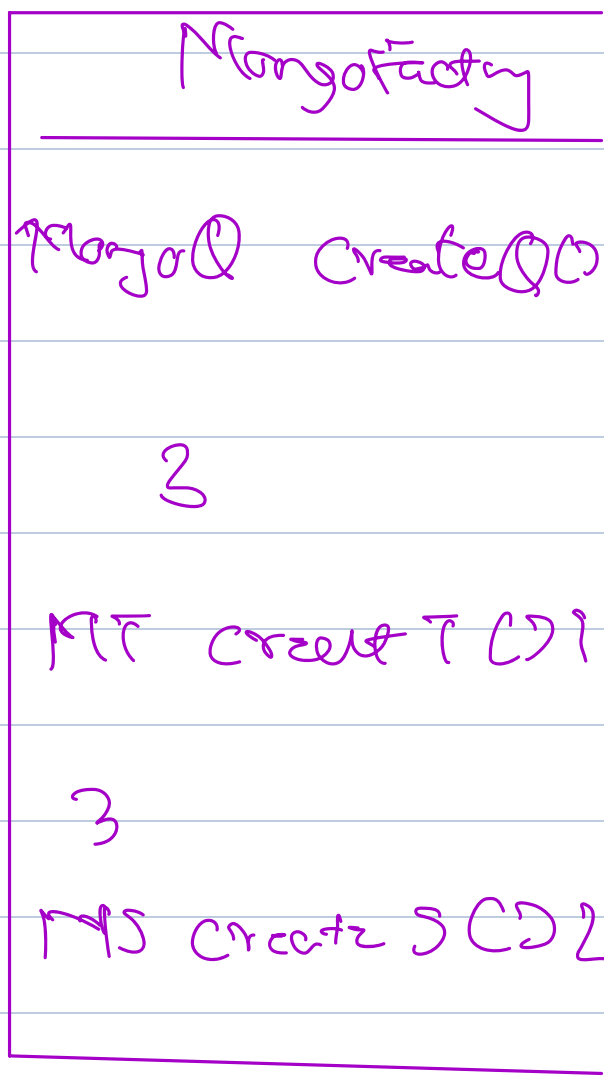
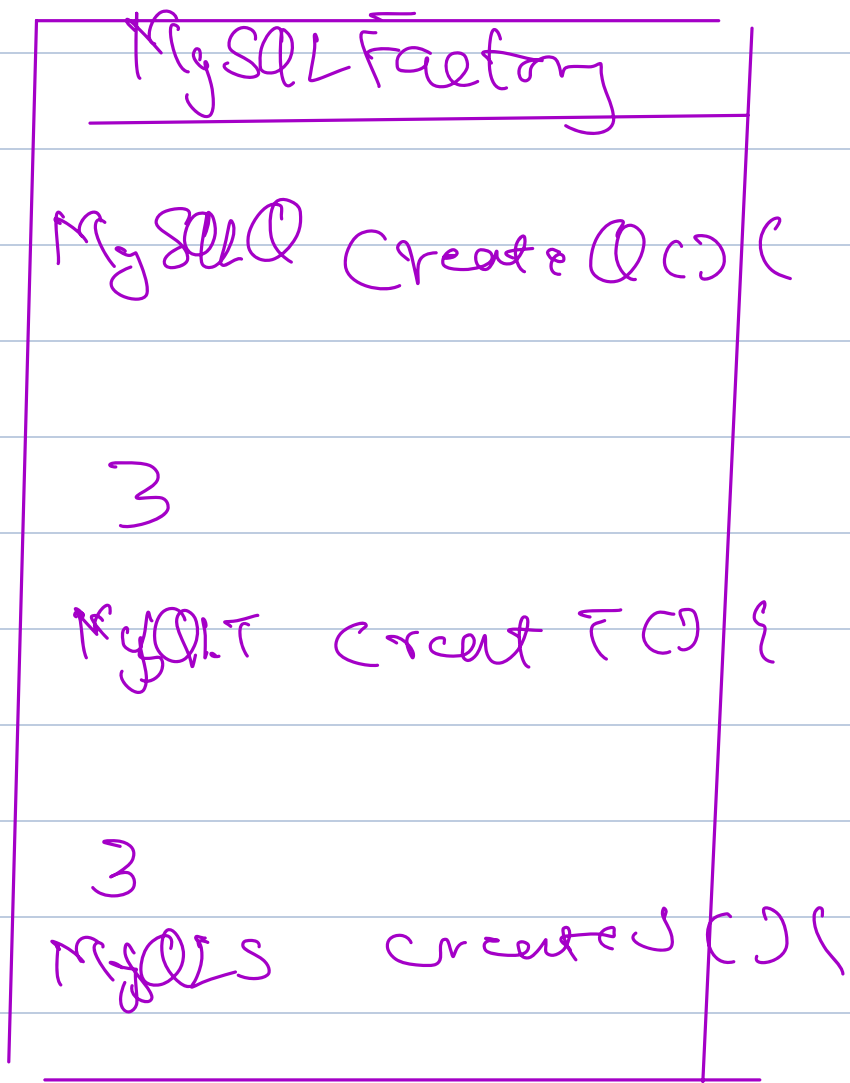
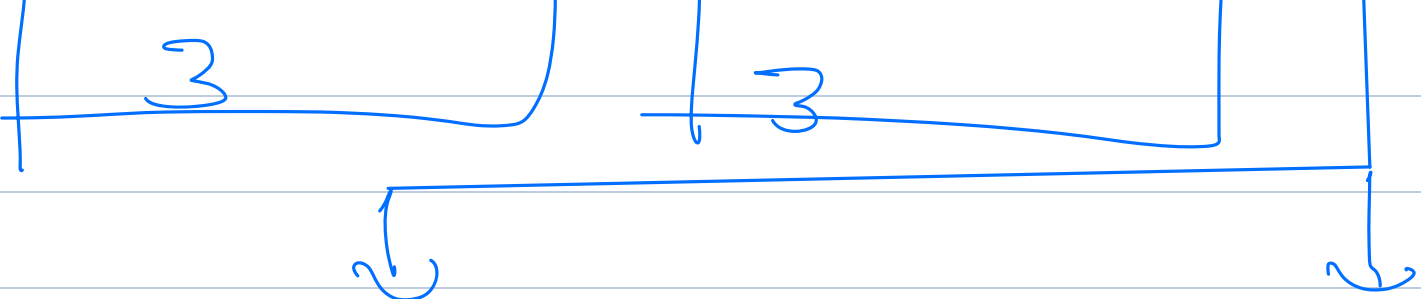
```
        q.execute();
```

```
}
```

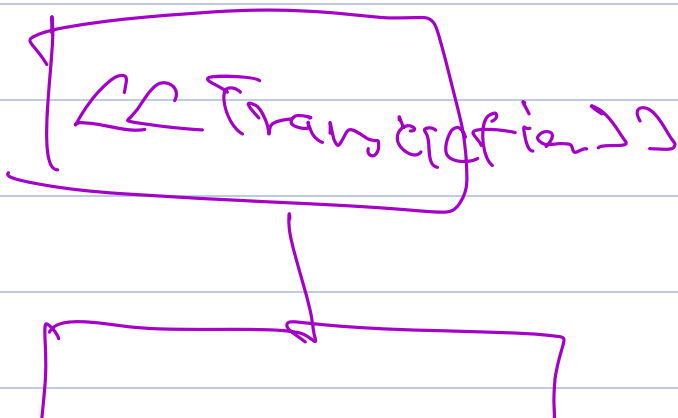
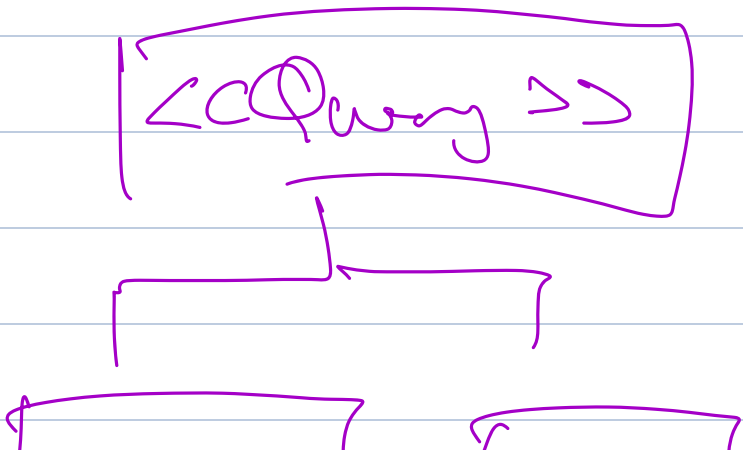
3

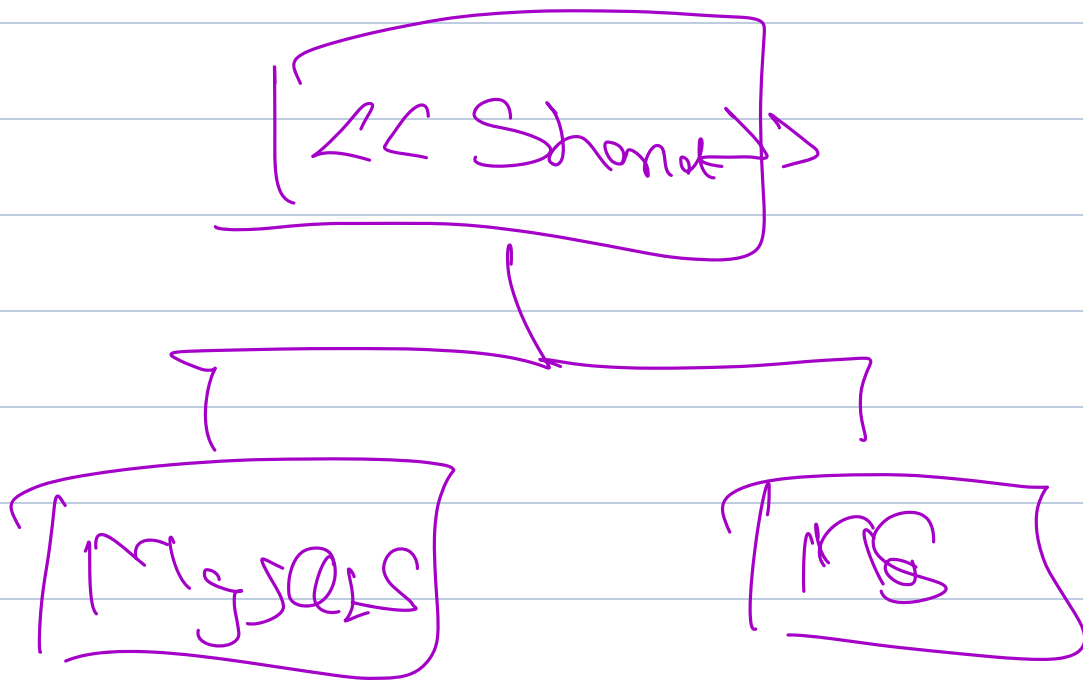
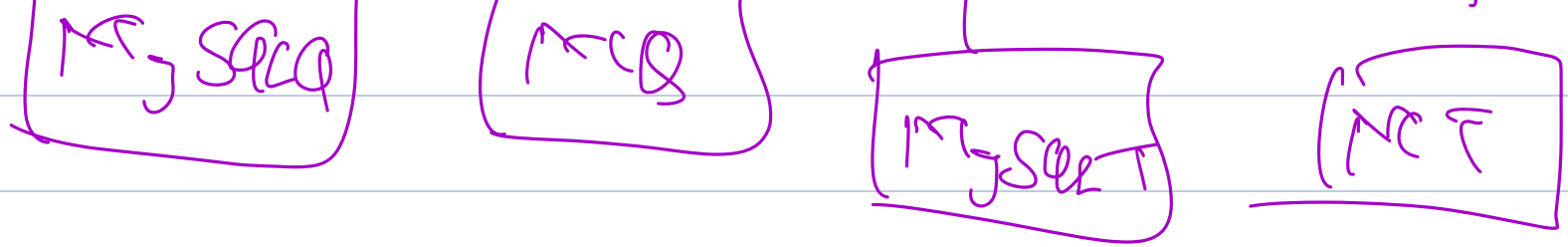
3 Final design ~





3





Updated client code:

~~DelEdBase~~ db;

public User saveUser(^{User u}) {

DBF dbf = clb.createDBF();

Query q = dbf.createQuery();

3

9 - executed;

Practical Example

class Flutter {

```
Menu CreateMenu() {  
    if (platform == "IOS") {  
        return new IOSMenu();  
    } else {  
        return new AndroidMenu();  
    }  
}
```

3

Button CreateButton () {

if (...) {

;

}

else {

;

}

}

Carousel CreateCarousel () {

if (...) {

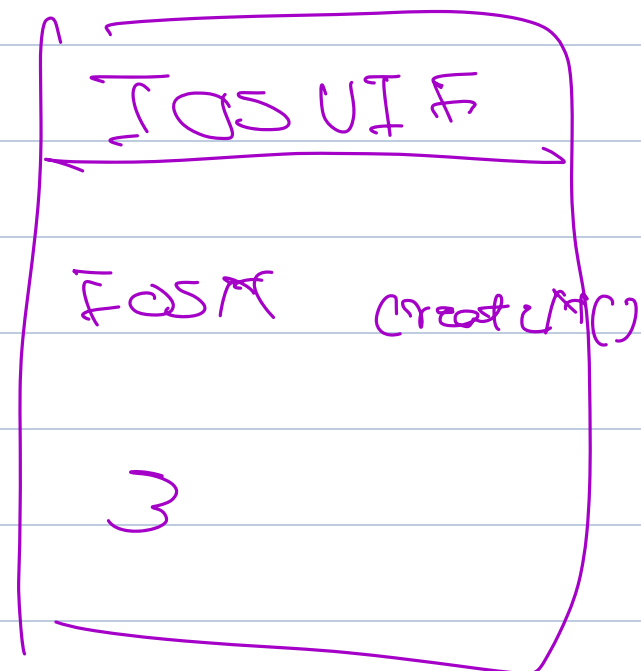
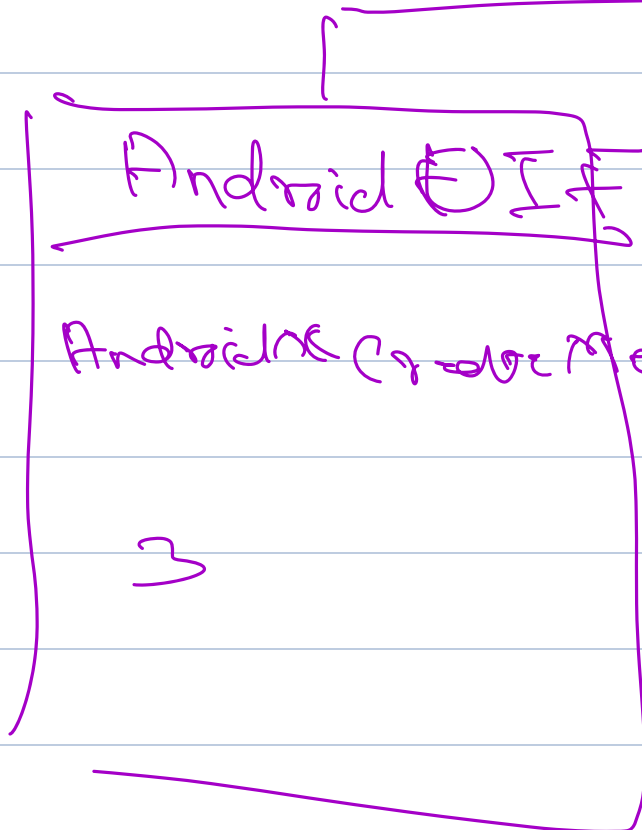
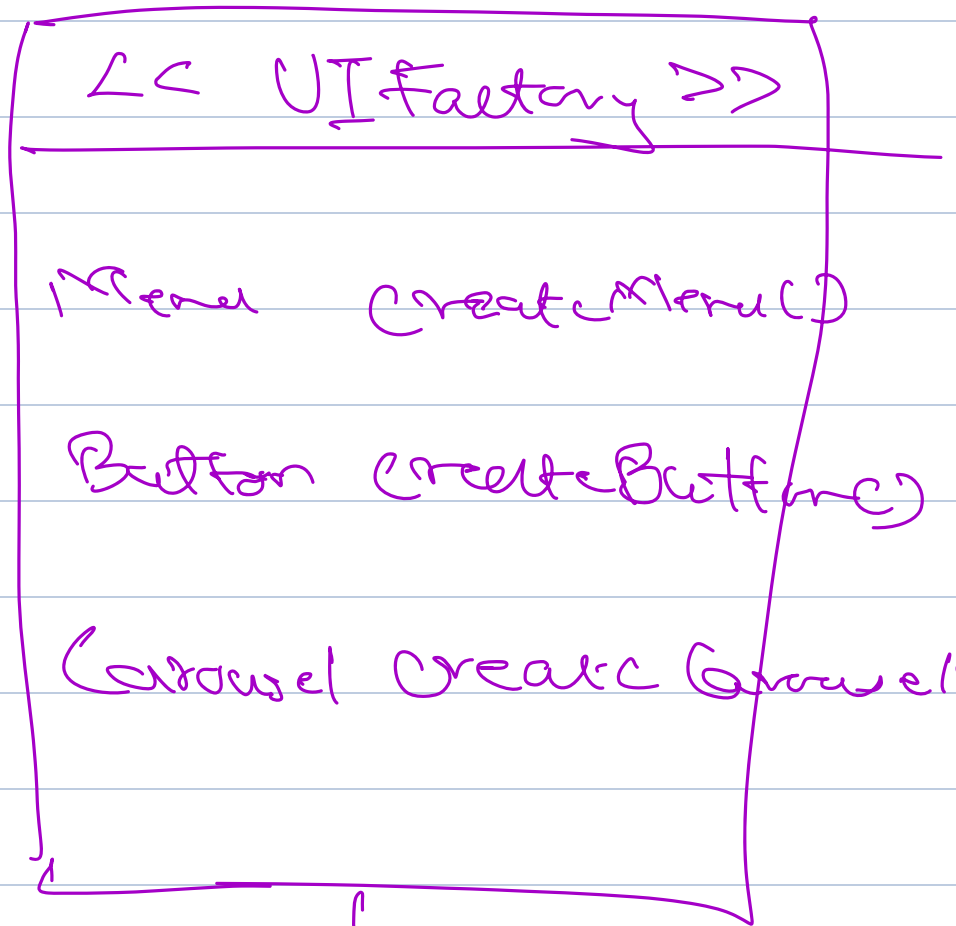
;

else {

}

}

→



3 ways to create an Obj of
UIF

① Hardcoding

class Flutter {

UIF uif = new UIFactory();

3

② Dependency Injection

class Flutter {

public Flutter(UIF uif) {

 this.uif = uif

}

}

Client:

Flutter F = new Flutter(
 new AF());

③ We create a method for this which either takes an input from client or work with an environment variables.

```
System.getenv("    ");
```

```
class Flutter {
```

```
    UIFactory get UIFactory (String  
                                platform) {
```

```
        if (platform.equals("IOS") {
```

```
            return new IOSFactory();
```

```
        }
```

```
        else {
```

```
            return new AF();
```

3

Client:

```
Flutter f = new Flutter();
```

```
IOSFeeder ios = f.getUIFactory("IO");
```

```
ios.createButton();
```

Pros: ① flexible

Summary:

① 1 & only 1 obj throughout
entire code base: Singleton

② Lots of attrs
Some need validatⁿ
Some^{attr} optional & some compulsory

Builder

③ Obj creatⁿ is expensive:
Prototype

Registry: used to store
prototypes

④ Factory Method: delegatⁿ of obj creatⁿ from client to relevant class.

Abstr factory: collectⁿ of FAB

① Look ↯ Time

② Stack with prev modules.
↙

DSA

SQL ← Upcoming Mock

↑
Revision

Next Sat is last class
for LLD 2 $\rightarrow$ 24th Feb

Aldt you 2nd Mar
have
100% PSP
in LLD2

Every class will have goal $\rightarrow$ PO

Complete SOLID-1 Δ SOLID-2
assignment

$\rightarrow$ 30 - 45 mins

Singleton assignment